

Analysis on Accelerating Object Detection in Edge Device with Half-Precision Floating Point

TAEKOAN YOO
Embedded Software Lab.

- **Object detection using deep learning**
 - Applications: obstacle detection, vehicle distance maintenance
 - Deep learning achieves high object detection accuracy.
 - Limitation: Most approaches rely on cloud-based systems equipped with high-performance GPUs.
 - Edge devices: mobile APs (Application Processors)
 - Low power consumption, but limited processing performance.
 - Real-time : 1080p/60fps **[ref]**
 - Achieving 60 FPS (approximately 0.02 s/frame) is challenging on edge devices.



[ref] *Mobile Machine Learning Hardware at ARM: A Systems-on-Chip (SoC) Perspective (arXiv'18)*

– Contribution

- Use the **Arm Compute Library (ACL)**.
 - Leverage **SIMD (Single Instruction, Multiple Data)** architectures on CPUs and GPUs.
- Investigate whether object detection can be accelerated toward **real-time performance while maintaining accuracy (mAP)**.

Compute Library

A software library for computer vision and machine learning

The Compute Library is a collection of low-level functions optimized for Arm CPU and GPU architectures targeted at image processing, computer vision, and machine learning. It is available free of charge under a permissive MIT open source license.

Overview

4

15

- **0. Background**
- **1. ACL : NEON**
- **2. ACL : OpenCL**
- **3. Future work**
- **Conclusion**

0. Background

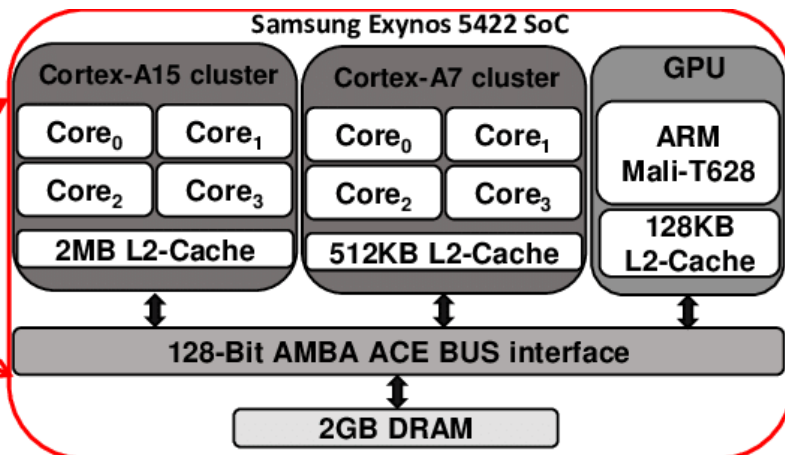
5

15

- Recent object detection inference time

- dataset : KITTI
- device : Odroid XU3 (ARMv7)
- The number of threads : 4

Object Detection NN	Inference time
YOLOv2	18.84s
YOLOv2-tiny	4.01s
SSD	14.88s
SqueezeDet	1.658s
SqueezeDet+	4.67s`



0. Background

6

15

- **Arm Compute Library(ACL)**
 - A library for optimized computation on Arm devices.
 - Acceleration Methods
 - NE version: Uses NEON to exploit the CPU's SIMD architecture.
 - CL version: Uses OpenCL for GPU acceleration.
 - Baseline: Original SqueezeDet object detection
 - using 32-bit floating-point operations with NEON.

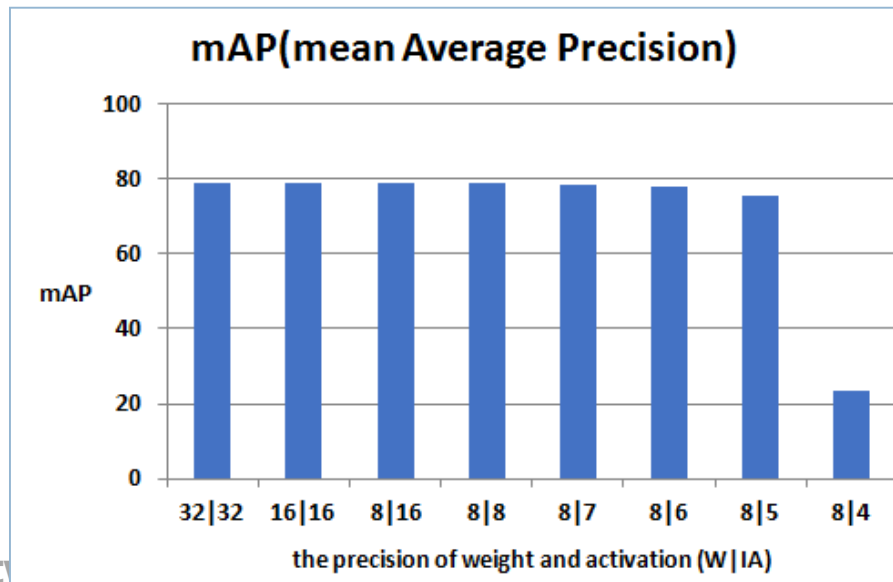
0. Background

7

15

- **mAP(mean Average Precision)**
 - Use a pretrained 32-bit floating-point SqueezeDet model.
 - mAP remains unchanged when using 16-bit floating-point computation.

W / IA precision	mAP
32 / 32	78.8
16 / 16	78.8
8 / 16	78.8
8 / 8	78.9
8 / 7	78.4
8 / 6	78.2
8 / 5	75.6
8 / 4	23.3



1. ACL : NEON

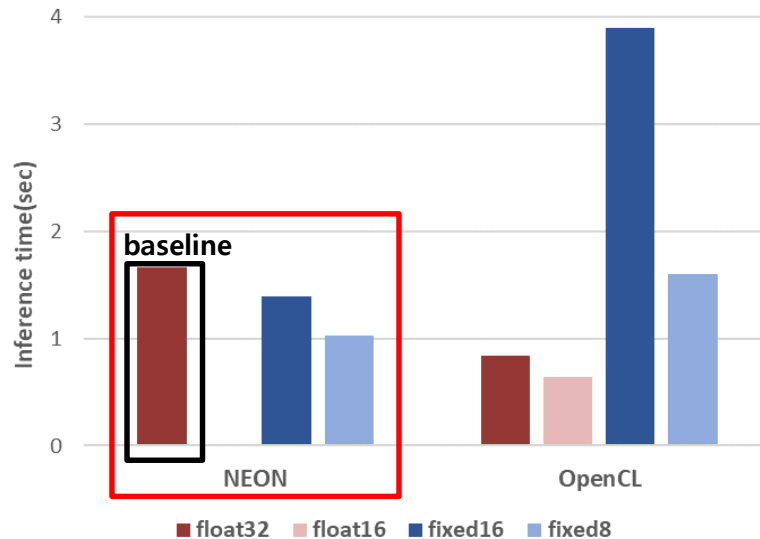
8

15

- **Experiment**

- 16-bit floating-point operations are not supported.
 - Supported starting from ARMv8.2 (Cortex-A75/A55).
- Fixed-point computation reduces inference time.

NEON	Inference time
float32	1.658 sec
float16	X
fixed8	1.393 sec
fixed16	1.029 sec



2. ACL : OpenCL

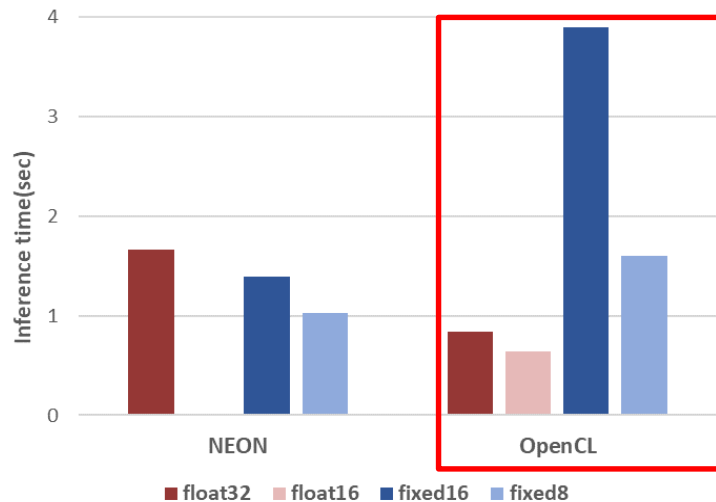
9

15

- **Experiment**

- Fixed point overhead
 - Fixed-point operations incur conversion overhead to efficiently utilize FMA units.

OpenCL	Inference time	Memory
float32	0.844 sec	833MB
float16	0.64 sec	423MB
fixed8	1.597 sec	
fixed16	3.898 sec	



2. ACL : OpenCL

10

15

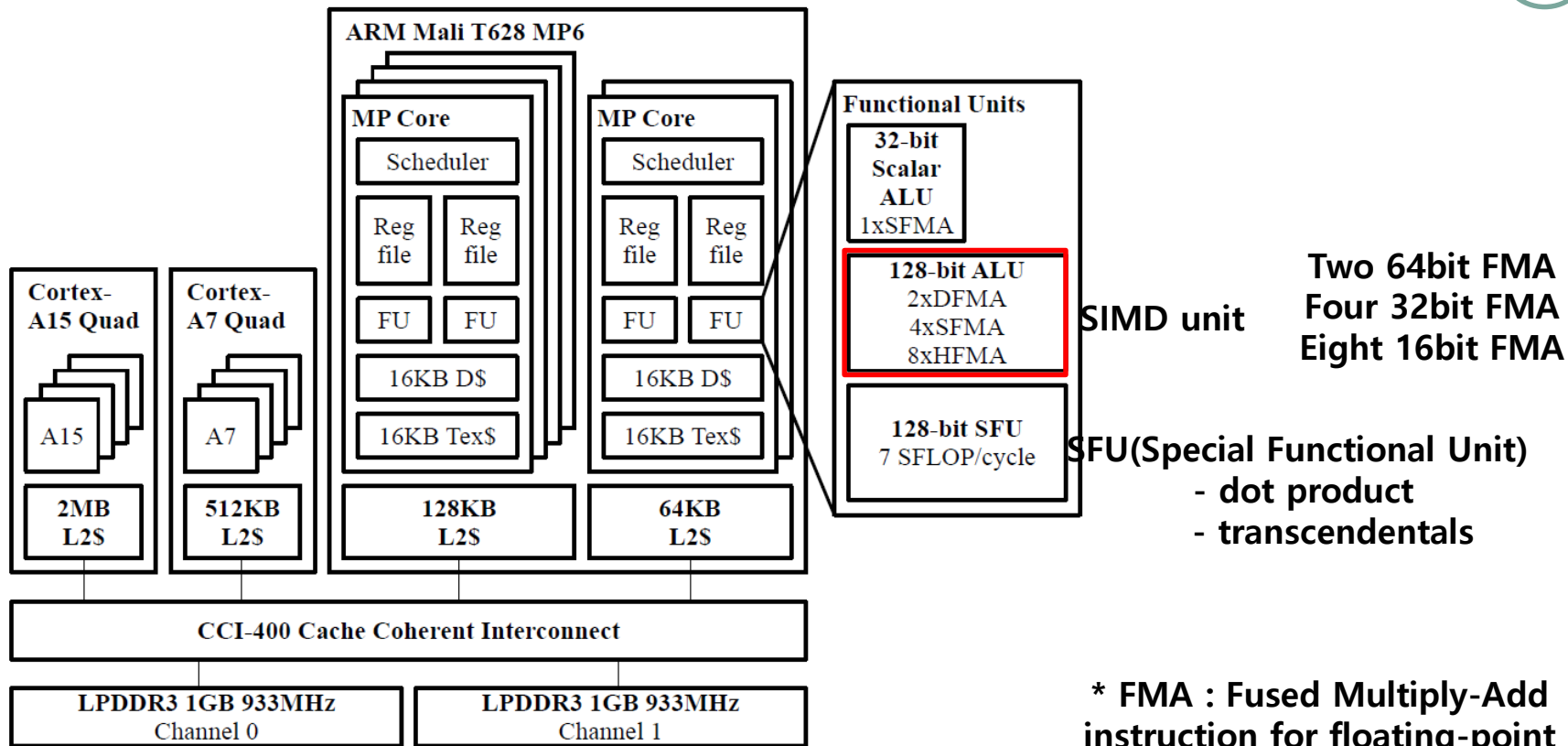


Figure 6.2: Samsung Exynos 5422 block diagram

* FMA : Fused Multiply-Add instruction for floating-point computation

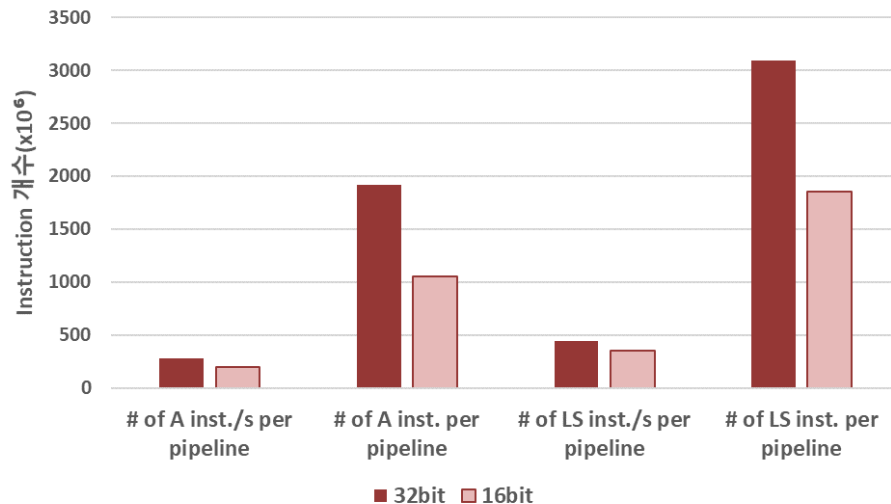
2. ACL : OpenCL

11

15

- **Changes in Instruction Count**

- Streamline Profiling Tool: A profiling tool provided by Arm.
 - A instruction : Number of arithmetic instructions
 - LS instruction : Number of load/store instructions



3. Future work

12

15

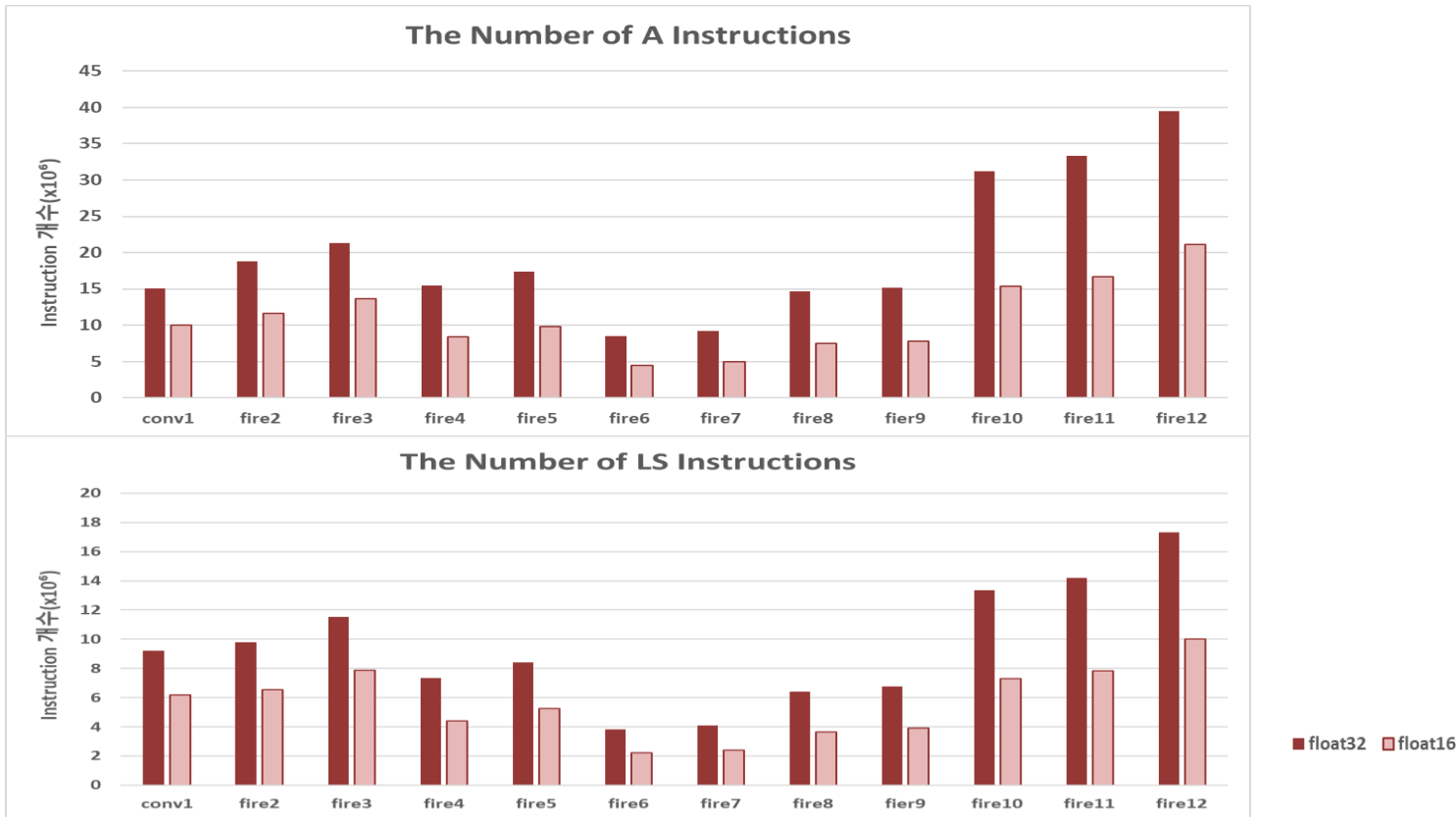
- **16bit floating point in NEON**
 - Half-precision floating point is not implemented in NEON on the current platform. Implementing it would enable a direct comparison with OpenCL acceleration.
- **Training with 16bit floating point**
 - The current model is converted from a pretrained 32-bit floating-point model. Training directly with 16-bit floating point could reveal its effect on mAP.
- **Using compression method**
 - Since the experiments use the original SqueezeDet architecture, existing compression and quantization techniques may further improve performance toward real-time inference.
- **Device-level optimization for 16-bit floating-point operation**

3. Future work

13

15

- A/LS Instruction Counts by Layer : depending on layer

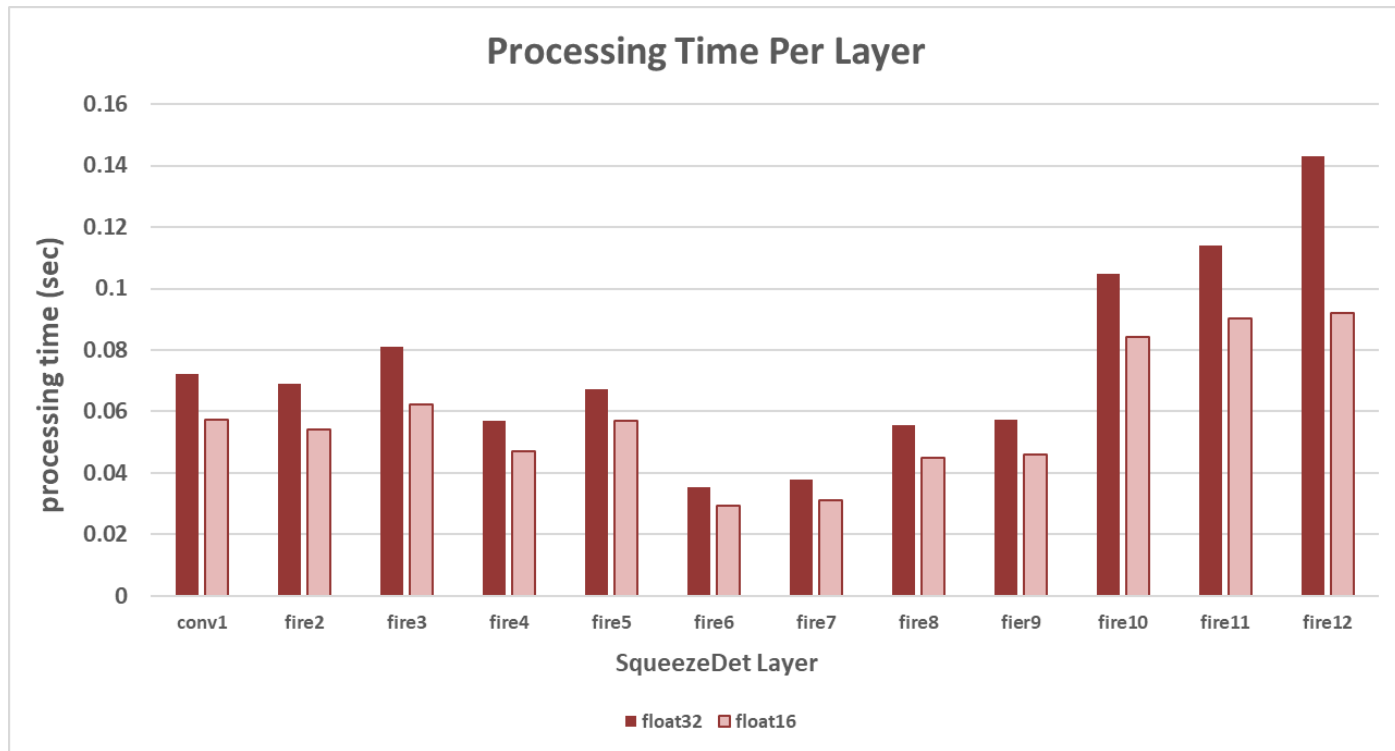


3. Future work

14

15

- **Processing Time : depending on layer**



- **Conclusion**

- We used the Arm Compute Library (ACL) to accelerate object detection on an Arm device toward real-time performance (0.02 s/frame).
- Using OpenCL in ACL provided a 50% speedup, while 16-bit floating-point computation improved inference speed by 62% over the original SqueezeDet baseline, achieving 0.64 s per frame.
- We analyzed GPU execution using the Streamline profiling tool and identified directions for further optimization toward real-time performance (0.02 s/frame).